Stop-and-Wait ARQ Protocol Simulation

ID - Card

Name: Srimoneyshankar Ajith
Roll_Number: 2024BCS0055
Date: 09-03-2026

Aim

To simulate the **Stop-and-Wait ARQ protocol** in C++ and observe the behavior of frame transmission, acknowledgements, timeouts, and retransmissions.

Theory

Stop-and-Wait ARQ is a **data link layer protocol** used for reliable data transmission between sender and receiver.

In this protocol:

- The sender transmits **one frame at a time**.
- After sending a frame, the sender waits for an **acknowledgement (ACK)** from the receiver.
- If the ACK is received successfully, the sender proceeds to send the **next frame**.
- If the frame is **lost during transmission**, the receiver does not send an acknowledgement.
- The sender then assumes a **timeout** has occurred and **retransmits the same frame**.

To distinguish between new frames and retransmissions, **sequence numbers (0 and 1)** are used alternately.

Source Code

C++

```cpp
#include <iostream>
using namespace std;

int main() {
    int n = 10;
    int total_packets = 0;
    int retransmissions = 0;
    int seq = 0;

    for(int i = 0; i < n; i++) {

        cout << "Sender: Sending Frame " << i << " (sequence number = " << seq << ")" << endl;
        total_packets++;

        if((i+1) % 4 == 0) { // simulate loss

            cout << "Timeout occurred. Retransmitting Frame " << i << " (sequence number = " << seq << ")" << endl<<endl;

            retransmissions++;
            total_packets++;

            cout << "Receiver: Frame " << i << " received successfully." << endl;
            cout << "Receiver: ACK " << seq << " sent" << endl;
        }
        else {
            cout << "Receiver: Frame " << i << " received successfully." << endl;
            cout << "Receiver: ACK " << seq << " sent" << endl;
        }

        seq = 1 - seq; // toggle sequence number
    }

    cout << "Total packets transmitted: " << total_packets << endl;
    cout << "Total frames retransmitted: " << retransmissions << endl;

    return 0;
}
```

Output

```
■ money on Aether via ⚙ v6.3.0 at …\College\4th Semester\ICS224 ❯ ./a.exe
Sender: Sending Frame 0 (sequence number = 0)
Receiver: Frame 0 received successfully.
Receiver: ACK 0 sent
Sender: Sending Frame 1 (sequence number = 1)
Receiver: Frame 1 received successfully.
Receiver: ACK 1 sent
Sender: Sending Frame 2 (sequence number = 0)
Receiver: Frame 2 received successfully.
Receiver: ACK 0 sent
Sender: Sending Frame 3 (sequence number = 1)
Timeout occurred. Retransmitting Frame 3 (sequence number = 1)

Receiver: Frame 3 received successfully.
Receiver: ACK 1 sent
Sender: Sending Frame 4 (sequence number = 0)
Receiver: Frame 4 received successfully.
Receiver: ACK 0 sent
Sender: Sending Frame 5 (sequence number = 1)
Receiver: Frame 5 received successfully.
Receiver: ACK 1 sent
Sender: Sending Frame 6 (sequence number = 0)
Receiver: Frame 6 received successfully.
Receiver: ACK 0 sent
Sender: Sending Frame 7 (sequence number = 1)
Timeout occurred. Retransmitting Frame 7 (sequence number = 1)

Receiver: Frame 7 received successfully.
Receiver: ACK 1 sent
Sender: Sending Frame 8 (sequence number = 0)
Receiver: Frame 8 received successfully.
Receiver: ACK 0 sent
Sender: Sending Frame 9 (sequence number = 1)
Receiver: Frame 9 received successfully.
Receiver: ACK 1 sent
Total packets transmitted: 12
Total frames retransmitted: 2
```

- Frames are sent sequentially from sender to receiver.
- Every **4th frame is simulated as lost**.
- The sender detects the absence of acknowledgement.
- A **timeout occurs**, and the frame is retransmitted.
- Once the receiver successfully receives the frame, it sends the corresponding **ACK**.

The final output also displays:

- **Total packets transmitted** = 12
- **Total frames retransmitted** = 2

Conclusion

The simulation successfully demonstrates the working of the **Stop-and-Wait ARQ protocol**.
It shows how acknowledgements, sequence numbers, and retransmissions ensure reliable data transfer even when some frames are lost during transmission.